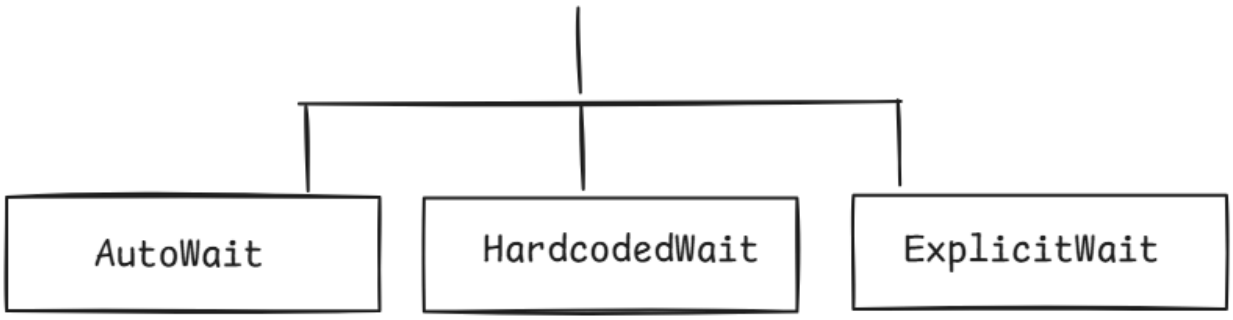
Wait Statement

A wait statement is a mechanism in automation testing that pauses the execution of a test script until a specific condition is met or a timeout period is reached.
--In real-time application
Web elements may take some time to appear, become visible, become enabled, or update. If the script acts before the element is ready, the test may fail, so waits keep the script in sync with the web application.

Why we use waits in Playwright

--> Web elements load dynamically after the page opens
--> An element may be in the DOM but not yet visible
--> A button may be disabled at first and become enabled later
--> Page navigation or URL change takes time
--> Data loads in the background through API (AJAX) calls
--> Loaders, spinners, and popups appear and disappear
--> To avoid flaky tests

Types of wait



1)Autowait
Auto-wait in Playwright means that Playwright automatically waits for an element to be ready before performing an action on it, so manual waits are not needed in most cases. It is also known as a smart wait or a built-in wait.
Before performing an action such as click or fill, Playwright runs a range of actionability checks to make sure the action behaves as expected. It checks that the element is attached to the DOM, visible, stable,enabled, able to receive events,and editable
The default timeout for auto-waiting is 30 seconds.

if we want to change the default timeout for auto waits

CASE 1: Change the default timeout for the whole test block

If we want to change the default timeout for the whole test block, we use setDefaultTimeout().

```
Syntax:page.setDefaultTimeout(millisec);
```

CASE 2: Change the timeout for one particular method only

If we want to change the timeout for one particular method only, we pass the timeout inside that method.

```
Syntax:await page.locator("xpath").fill("value", { timeout: millisec });
```

CASE 3: Change the timeout for all the test blocks in one file

If we want to change the timeout for all the test blocks in one specific file, we use test.use(). Write it after the imports and before the test blocks.

```
Syntax:test.use({ actionTimeout: millisec });
```

CASE 4: Change the timeout for all the files in the project

If we want to change the timeout for all the test files in the project, we set actionTimeout in the config file. Go to playwright.config.ts and add it inside the use block.

```
Syntax:
       use: {
            actionTimeout: millisec
            }
```

AutoWait.spec.ts

```ts
import {test} from "@playwright/test"

// CASE 3: auto wait timeout of 25 seconds for all the tests in this file
test.use({ actionTimeout: 25000 })
Run | Debug | Inspect
test.only("autowait1", async ({ page }) => {
  // CASE 1: auto wait timeout of 40 seconds for all the methods in this test
  page.setDefaultTimeout(40000)
  await page.goto("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login")
  await page.locator("//input[@placeholder='Username']").fill("Admin")
  await page.locator("//input[@name='paword']").fill("admin123")

  // CASE 2: auto wait timeout of 1 second for this click only
  await page.locator("//button[@type='submit']").click({ timeout: 1000 })
})

Run | Debug | Inspect
test("autowait2", async ({ page }) => {
  await page.goto("https://www.saucedemo.com/")
  await page.locator("#user-name").fill("standard_user")
  await page.locator("[type='password']").fill("secret_sauce")
  await page.locator(".submit-button").click()
  await page.locator("//div[text()='Sauce Labs Backpack']").click()
})
```

playwright.config.ts

```ts
export default defineConfig({
  reporter: 'html',
  /* Shared settings for all the projects below. See https://playwright.dev/docs/api/class-tes
  use: {
    actionTimeout:25000,
    testIdAttribute:'data-test',
    /* Base URL to use in actions like `await page.goto('')`. */
    // baseURL: 'http://localhost:3000',

    /* Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-view
    trace: 'on-first-retry',
  },
```

2) Hard-coded wait
it is used to pause the test execution for a fixed amount of time.
Playwright stops the execution and waits for the exact time you specify, even if the element is ready earlier.

syntax: await page.waitForTimeout(milliseconds);

TS hardwait.spec.ts

tests > TS hardwait.spec.ts > ...

```ts
import {test} from "@playwright/test"
Run | Debug | Inspect
test("hardcodedwait",async({page})=>{
 await page.goto
 ("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login")//1 sec
 await page.waitForTimeout(2000)
 await page.locator("//input[@placeholder='Username']").fill("Admin")
  await page.waitForTimeout(2000)
 await page.locator("//input[@name='password']").fill("admin123")
  await page.waitForTimeout(2000)
 await page.locator("//button[@type='submit']").click()
})

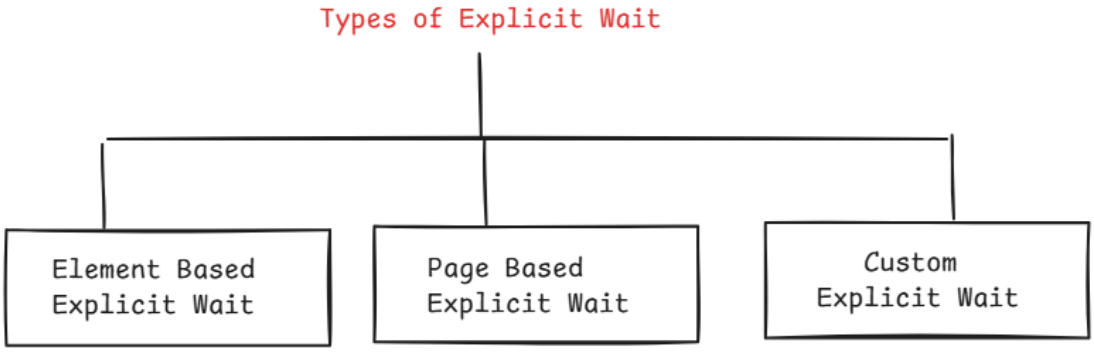
Run | Debug | Inspect
test.only("hardcodewait1",async({page})=>{
 await page.goto("https://www.amazon.in/")
 await page.locator("input#twotabsearchtextbox").fill("shoes")
 await page.waitForTimeout(2000)
 let autosuggestion=await
 page.locator("//div[@role='row']").allInnerTexts()
console.log(autosuggestion)
})
```

Disadvantages of hard-coded wait:

--> Slows down performance
--> Increases execution time because of the fixed wait
--> Causes flakiness
--> Not flexible

3)Explicit wait

It Pausing the script until a specific condition is met before continuing. Playwright proceeds as soon as the condition is true, and throws a TimeoutError if it isn't met within the timeout.



1)Element Based Wait: Element-Based Waits: Waiting for a particular element, its text, or its state before performing the next action.

a)wait for text

Waiting until a specific text becomes visible on the page before performing the next action

```
syntax:await page.locator("#value",{hasText: 'textvalue'}).waitFor();
```

b)wait for timeout

Wait for an Element with a Specified Timeout

```
syntax:await page.locator("#value").waitFor({timeout:1000});
```

c)wait for state

Waits until an element reaches a specified state before the script continues.

States:
visible: the element is present in the DOM and shown on screen
hidden: the element is not visible (hidden with CSS) or not present in the DOM
attached: the element is present in the DOM (it may not be visible)
detached: the element is removed from the DOM

```
syntax:await page.locator("#idvalue").waitFor({ state: 'visible' });
```

d)waitforSelector

Waits until an element matching the given selector (CSS, XPath, or text) reaches a specified state before the script continues. By default it waits for the element to become visible.

```
syntax:await page.waitForSelector('#idvalue');
```

TS Explicitwait.spec.ts

tests > TS Explicitwait.spec.ts > test("elementBasedWait") callback

```ts
import {expect, test} from "@playwright/test"
Run | Debug | Inspect
test("elementBasedWait",async({page})=>{
await page.goto("https://www.amazon.in/")
await page.locator("input#twotabsearchtextbox").fill("shoes")
//elementbasedwait-text
//  await page.locator("//div[@role='row']",{hasText:'for woman'}).waitFor()

//elementbasedwait-timeout
// await page.locator("//div[@role='row']").first().waitFor({timeout:1000})

//elementbasedwait-state
//  await page.locator("//div[@role='row']").first().waitFor({state:'visible'})

//elementbasedwait-selector
await page.waitForSelector("//div[@role='row']")
let autosuggestion:string[]= await page.locator("//div[@role='row']").allInnerTexts()
console.log(autosuggestion)
})
```

2)Page Based wait

Waiting for a page-level event, such as navigation, a URL change, a page load state, or network activity.These waits ensure that the page is fully loaded and ready before Playwright performs the next action.

a)waitforNavigation

It is used to wait until the page navigates to a new page or URL. Navigation happens when you click a link, click a button, or submit a form. Playwright waits for the navigation to finish before performing the next action, so it doesn't try to interact with the new page too early, which can cause test failures.

```ts
Run | Debug | Inspect
test("pagewait-navigation",async({page})=>{
    await page.goto("https://the-internet.herokuapp.com/login")
    await page.locator("#username").fill("tomsmith")
    await page.locator("#password").fill("SuperSecretPassword!")
    await Promise.all([
        page.waitForNavigation(),
        page.locator("[type='submit']").click()
    ])
    await expect(page).toHaveURL(/secure/)

})
```

b)waitforLoadState

It is used to pause the script until the page reaches a specific load state. It is helpful when you want to make sure the page is ready before interacting with elements, especially after navigation, form submission, or JavaScript-based updates.
The most commonly used load states are:
"domcontentloaded" → waits until the HTML document is loaded
"load" → waits until the entire page and its resources are loaded (default)
"networkidle" → waits until all network requests are finished

```
Run | Debug | Inspect
test("pagewait-loadstate",async({page})=>{
    await page.goto("https://www.amazon.in/")
    await page.locator("#nav-cart").click()
    // await page.waitForLoadState('domcontentloaded')
   // await page.waitForLoadState('load')
   await page.waitForLoadState('networkidle')
    await expect(page).toHaveURL(/cart/)
})
```

c)waitforEvent

It is used to wait until a specific event happens on the page or browser. Events are actions such as a new tab opening, a file downloading, or a popup appearing. It makes Playwright wait for the event before continuing.

eventName is the name of the event that Playwright should wait for, such as 'popup' or 'download'.

```
Run | Debug | Inspect
test.only("pagewait-event",async({page})=>{
    await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
    await page.locator("#writeArea").fill("hi ,im want download a file")
 let [downloadfile] = await Promise.all([
        page.waitForEvent('download'),
        page.locator("#downloadButton").click()
    ])
     console.log(await downloadfile.path())
})
```

3) custom wait

It is used when we want Playwright to wait until a user-defined condition becomes true before continuing test execution. Playwright provides the page.waitForFunction() method to wait for a custom JavaScript condition.

waitForFunction()

It is used to pause the test script until a custom JavaScript condition inside the browser returns true. It is helpful when waiting for something that doesn't have a fixed selector or involves dynamic DOM changes.

Syntax:

```
await page.waitForFunction(() =>{
 return condition;
});
```

```
Run | Debug | Inspect
test.only("customwait",async({page})=>{
    await page.goto("https://www.amazon.in/")
    await page.locator("#nav-cart").click()
    await page.waitForFunction(()=>{
        return document.readyState==='complete'
    })
})

Run | Debug | Inspect
test("custom",async({page})=>{
 await page.goto("https://www.amazon.in/")
 await page.locator("input#twotabsearchtextbox").fill("shoes")

  await page.waitForFunction(()=>{
      let ele= document.querySelectorAll(".s-suggestion-container")
      ele.length>1
    })
let autosuggestion:string[]= await page.locator("//div[@role='row']")
.allInnerTexts()
console.log(autosuggestion)
})
```